

one-dimensional array

Definition

- A **one-dimensional** array is a data structure **identified by a variable** and representing **a sequence of elements (values) of the same type**.
- An element can be accessed **by the array identifier (array name)** and **its position** in the array named **index**.



T is a **one-dimensional** array of **size N**

T is a **vector** of **size N**.

Utility

- An array is a data structure made up of a finite number of elements of the same type.
- When you want to represent several items of data of the same type with a single variable, you use an array.
- **Example**
- a program that processes student grades (N grades, N known in the program) => use an array

T_Notes	14	12.5	...	9	18.25
	1	2		N-1	N

Characteristics of an array

- **Characteristics**

- **array_name**: The identifier for the array.
- **data_type**: Specifies the type of elements the array will hold (Integer, real, character,...).
- **array_size** :positive integer representing the number of elements in the array.

Declaration of an array

- **Déclaration**

Nom_tab : Array [size] of data_type

- **Examples**

T1 : array [50] of integer;

T2 : array [20] of real;

T3 : array [20] of character;

- **Note**

It is also possible to define a array type, as in the following example:

```
CONST
    maxSize = 50
TYPE
    Array_int : array [maxSize] of integer ;
VAR
    T : Array_int;
```

Declaration d'un array en C

- **Déclaration**

```
data_type array_name [size];
```

- **Note:** The element indexes range from 0 to (size-1),

- **Examples:**

```
int T1[50];  
  
int taille=255; int T11[taille];  
  
float T2[20];  
  
char T3[20];
```

Accessing elements in an array

- To access any element in an array, use the following syntax:

`arr_name [element_index ]`

- Example

T	100	90	80	70	60
indices	1	2	3	4	5
contenu	T[1]	T[2]	T[3]	T[4]	T[5]

Accessing elements in an array

Example :

A few operations on arrays :

```
1  Int t1 [4] ;
2  int x=5 ;
3  t1[3]=-1 ;
4  t1[x]=2 ;
5  t1[(x*2)-8]=4 ;
6  int y = 2;
7  t1[1] = t1[2]-9;
8  x = t1[3]-t1[2];
9  t1[2] = t1[1]+ t1[3];
10 t1[0] = x + y;
```

1	t1	<table><tr><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>					0	1	2	3						
0	1	2	3													
2	t1	<table><tr><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>					0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x			
0	1	2	3													
5																
x																
3	t1	<table><tr><td></td><td></td><td></td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>				-1	0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x			
			-1													
0	1	2	3													
5																
x																
4	t1	<table><tr><td></td><td></td><td></td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>				-1	0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x			
			-1													
0	1	2	3													
5																
x																
5	t1	<table><tr><td></td><td></td><td>4</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>			4	-1	0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x			
		4	-1													
0	1	2	3													
5																
x																
6	t1	<table><tr><td></td><td></td><td>4</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>			4	-1	0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x	<table><tr><td>2</td></tr><tr><td>y</td></tr></table>	2	y
		4	-1													
0	1	2	3													
5																
x																
2																
y																
7	t1	<table><tr><td></td><td>-5</td><td>4</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>		-5	4	-1	0	1	2	3	<table><tr><td>5</td></tr><tr><td>x</td></tr></table>	5	x	<table><tr><td>2</td></tr><tr><td>y</td></tr></table>	2	y
	-5	4	-1													
0	1	2	3													
5																
x																
2																
y																
8	t1	<table><tr><td></td><td>-5</td><td>4</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>		-5	4	-1	0	1	2	3	<table><tr><td>-5</td></tr><tr><td>x</td></tr></table>	-5	x	<table><tr><td>2</td></tr><tr><td>y</td></tr></table>	2	y
	-5	4	-1													
0	1	2	3													
-5																
x																
2																
y																
9	t1	<table><tr><td></td><td>-5</td><td>-6</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>		-5	-6	-1	0	1	2	3	<table><tr><td>-5</td></tr><tr><td>x</td></tr></table>	-5	x	<table><tr><td>2</td></tr><tr><td>y</td></tr></table>	2	y
	-5	-6	-1													
0	1	2	3													
-5																
x																
2																
y																
10	t1	<table><tr><td>-3</td><td>-5</td><td>-6</td><td>-1</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>	-3	-5	-6	-1	0	1	2	3	<table><tr><td>-5</td></tr><tr><td>x</td></tr></table>	-5	x	<table><tr><td>2</td></tr><tr><td>y</td></tr></table>	2	y
-3	-5	-6	-1													
0	1	2	3													
-5																
x																
2																
y																

Reading the elements of an array

- Reading an element from an integer array T is similar to reading an integer variable.
- For an integer array T of size 10, the elements of the array are read as follows:

Reading the 1st element : **read(T[1]) ;**

..

Reading the i th element : **read(T[i]) ;**

..

Reading the 10 th element : **read(T[10]) ;**

Reading the elements of an array - Algo

- **Example:** write an algorithm to read N elements in an array, where N is equal to 100 at most.

```
Algorithm Array_read;
var
    i , N          : integer;
    T : array [100] of integer;
begin
    repeat
        write("Enter the number of elements in the array (1-100) :");
        read(N);
    until N > 0 AND N <= 100;

    write("filling the array:");

    for i ← 1 to N do
        write(" T[", i , "]:");
        read(T[i]);
    end;
end.
```

Reading the elements of an array – C language

- **Example:** write an program to read N elements in an array, where N is equal to 100 at most.

```
#include <stdio.h>
int main() {
    int n; // Demander à l'utilisateur le nombre d'éléments à read

    do{
        printf("Entrez le nombre d'éléments du tableau à read (N>0) : ");
        scanf("%d", &n);
    } while (n < 1); // boucler while le nombre d'éléments à read n'est pas valide

    //déclaration d'un array d'integer taille N

    int T[n];

    // read les N éléments du array

    printf("Remplissage du array :\n");
    for (int i = 0; i < n; i++) {
        printf("T[%d]: ", i);
        scanf("%d", &T[i]);
    }
}
```

Example of execution :

```
Entrez le nombre d'éléments du tableau à lire (N>0) : -4
Entrez le nombre d'éléments du tableau à lire (N>0) : 6
Remplissage du tableau :
T[0]: 4
T[1]: 125
T[2]: -24
T[3]: 187
T[4]: 501
T[5]: -12
```

Displaying the elements of an array

- Writing an element of an integer array T is similar to writing an integer variable.
- For an integer array T of size 10, the elements of the array are displayed as follows:

write 1st element: `write(T[1]);`

..

write ith element: `write(T[i]);`

..

write 10th element: `write(T[10]);`

Displaying the elements of an array

- **Example:** assuming that an integer array T of size N is stored in memory, write an Algorithm that allows you to display the elements of T as shown in the following example:

T	5	-9	15	21	0	-88	15	..	
	1	2	3	4	5	6	7		100

N= 7

Display:

[5, -9, 15, 21, 0, -88, 15]

Displaying the elements of an array

```
Algorithm Affichage_array;
var
  i , N           : integer;
  T : array [100] of integer;
begin
  // ici se trouve les instructions permettant de remplir le array T
  // ...
  write("array T:" );
  write("[");
  for i ← 1 to N do
    if i = N then
      write(T[i] , "]" );
    else
      write(T[i] , ", " );
    end ;
  end;
end.
```

Displaying elements of an array – C language

```
#include <stdio.h>
int main() {
    int n;
    do{
        printf("Enter the array size (N>0) : ");
        scanf("%d", &n);
    } while (n < 1); // loop as long as the number of elements to read is <=0

    int T[n]; //

    printf(" Filling the array :\n");
    for (int i = 0; i < n; i++) {
        printf("T[%d]: ", i);
        scanf("%d", &T[i]);
    }

    // Display array elements
    printf("\n Array T :\n");
    for (int i = 0; i < n; i++) {
        if(i==n-1) // display of last item
            { printf("%d", T[i]); }
        else // display of other elements
            { printf("%d, ", T[i]); }
    }
}
```

Example of execution :

```
Enter the array size (N>0) : 5
Filling the array :
T[0]: 7
T[1]: -5
T[2]: 12
T[3]: 3
T[4]: 14

Array T :
[7, -5, 12, 3, 14]
```

Sequential search - Objectif

Algorithm to search for **the 1st appearance** of a **value V** in an **array T**.

The algorithm returns the **position of v** if it exists in T and -1 otherwise.

T	-20	0	5	142	8	73	-4	5
	1	2	3	4	5	6	7	8

for **V = 8** The algorithm finds the value 8 and displays its position 5

for **V = 17** The algorithm displays that the value 17 does not exist

Sequential search in an array

search of $V=8$ in T

$V=8$

$i=1 \quad V = T[1] \Rightarrow 8 = -20 \quad \text{false}$

$i=2 \quad V = T[2] \Rightarrow 8 = 0 \quad \text{false}$

$i=3 \quad V = T[3] \Rightarrow 8 = 5 \quad \text{false}$

$i=4 \quad V = T[4] \Rightarrow 8 = 142 \quad \text{false}$

$i=5 \quad V = T[5] \Rightarrow 8 = 8 \quad \text{true}$

Stop because $v=T[i]$

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

We exit with **true** so 8 is found
at position **5**

Sequential search in an array

search of $V = -17$ in T

$V = -17$

$i=1 \quad V = T[1] \Rightarrow -17 = -20 \quad \text{false}$

$i=2 \quad V = T[2] \Rightarrow -17 = 0 \quad \text{false}$

$i=3 \quad V = T[3] \Rightarrow -17 = 5 \quad \text{false}$

$i=4 \quad V = T[4] \Rightarrow -17 = 142 \quad \text{false}$

$i=5 \quad V = T[5] \Rightarrow -17 = 8 \quad \text{false}$

$i=6 \quad V = T[6] \Rightarrow -17 = 73 \quad \text{false}$

$i=7 \quad V = T[7] \Rightarrow -17 = -4 \quad \text{false}$

$i=8 \quad \text{STOP, because } i > 7$

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

T	-20	0	5	142	8	73	-4
	1	2	3	4	5	6	7

On sort avec false donc -17 n'existe pas

Conclusion: the test $V = T[i]$ is repeated until :
 V is found or all elements of T are visited (i reaches the last index)

Sequential search in an array - Algorithm

Algorithm version #1

```
Algorithm sequential_search_array;
var
    i , N , V          : integer;
    T : array [100] of integer;
begin
    // Here are the instructions for filling in the T array.
    // .....
    // read the value V
    write("donner une valeur à rechercher dans T:" );
    read(V);
    // .....
    // sequential search for V in T
    i ← 1; trouve ← false;
    while trouve = false AND i ≤ N do
        if T[i] = V then trouve ← vrai; elsif;
        i ← i + 1;
    done;
    if trouve = vrai then
        write("The value ", V, " is located in T at position:", i );
    else
        write(" The value ", V, " does not exist in T ");
    elsif ;
end.
```

Sequential search in an array - Algorithm

Algorithm version #2

```
Algorithm sequential_search_array;
var
    i , N , V           : integer;
    T : array [100] of integer;
begin
    // Here are the instructions for filling in the T array.
    // .....
    // read the value V
    write("donner une valeur à rechercher dans T:" );
    read(V);
    // .....
    // recherche séquentielle de V dans T
    i ← 1;
    while T[i] <> V AND i <= N do
        i ← i + 1;
    done;
    if i <= N then
        write("The value ", v, " is located in T at position:", i );
    else
        write(" The value ", v, " does not exist in T ");
    endif ;
end.
```

Sequential search, C program

C Program version #1

```
#include <stdio.h>
int main() {
    int i, N, V;
    // remplissage du array T
    printf("Entrez la taille du array (N <= 100): "); scanf("%d", &N);
    int T[N];
    printf("Entrez les %d integers du array:\n", N);
    for (i = 0; i < N; i++) {
        printf("T[%d] = ", i); scanf("%d", &T[i]);
    }
    // lecture de V
    printf("donner une valeur à rechercher dans T: "); scanf("%d", &V);
    // recherche séquentielle de V dans T
    i = 0; int trouve=0;
    while (i < N && trouve==0) {
        if(T[i] == V){    trouve=1; }
        i++;
    }
    if (trouve=1) {
        printf("La valeur %d se trouve dans T à la position: %d\n", V, i);
    } else {
        printf("La valeur %d n'existe pas dans T\n", V);
    }
    return 0;
}
```

Sequential search, C program

Programme C version 2

```
#include <stdio.h>
int main() {
    int i, N, V;
    // remplissage du array T
    printf("Entrez la taille du array : "); scanf("%d", &N);
    int T[N];
    printf("Entrez les %d integers du array:\n", N);
    for (i = 0; i < N; i++) {
        printf("T[%d] = ", i); scanf("%d", &T[i]);
    }
    // lecture de V
    printf("donner une valeur à rechercher dans T: "); scanf("%d", &V);
    // recherche séquentielle de V dans T
    i = 0;
    while (i < N && T[i] != V) {
        i++;
    }
    if (i < N) {
        printf("La valeur %d se trouve dans T à la position: %d\n", V, i);
    }
    else {
        printf("La valeur %d n'existe pas dans T\n", V);
    }
    return 0;
}
```

Binary search

- **Goal:**

Determine the 1st position of a value in a sorted array

T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8

for **V= 8** Algorithm finds the value 8 AND displays its position 5

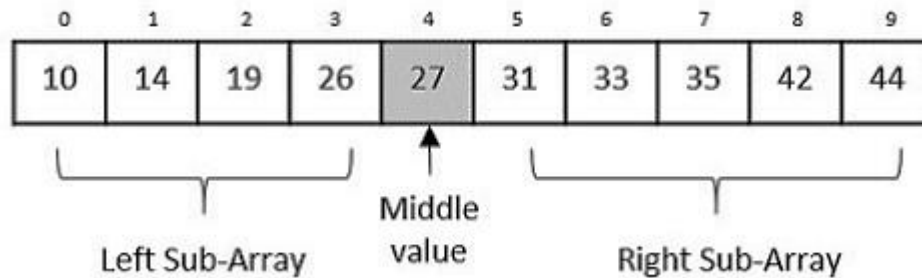
for **V= 3** Algorithm shows that the value 3 does not exist.

- **Note:**

Binary search works on **sorted arrays**.

Binary search

The principle is to split the T array into two sub-arrays.



In an **array T**, a search for a value **val** follows one of the three cases below:

- If **val = T[middle]** then **val** is found AND the search is complete.
- If **val < T[middle]** then we will search for **val** in the left part of array T.
- If **val > T[middle]** then we will search for **val** in the right part of array T.

The search continues as long as **T[middle]** is different from **val** and as long as the subarray dimension remains valid.

Binary search – example #1

Search of $V = 8$ in T

$V=8$

$\text{inf}=1$ $\text{sup}=8$

$m = (8+1)/2 = 4$

$\Rightarrow V > T[4] \Rightarrow (8 > 5) \Rightarrow \text{inf}=4+1=5$

$\text{inf}=5$ $\text{sup}=8$

$m = (5+8)/2 = 6$

$\Rightarrow V < T[6] \Rightarrow (8 < 73) \Rightarrow \text{sup}=6-1=5$

$\text{inf}=5$ $\text{sup}=5$

$m = (5+5)/2 = 5$

$\Rightarrow V = T[5] \Rightarrow (8 = 8) \Rightarrow$ Return the position **5**

Stop, because the value is found

	inf								sup		
T	-20	-4	0	5	8	73	142	201			
	1	2	3	4	5	6	7	8			
	m										

					inf			sup
T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8
					m			

	inf sup							
T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8
					m			

Binary search – example #2

Search of $V = 3$ in T

$V=3$

$bi=1$ $bs=8$ $m = (8+1)/2 = 4$

$\Rightarrow V < T[4] \Rightarrow (3 < 5) \Rightarrow bs=4-1=3$

$bi=1$ $bs=3$ $m = (3+1)/2 = 2$

$\Rightarrow V > T[2] \Rightarrow (3 > -4) \Rightarrow bi=2+1=3$

$bi=3$ $bs=3$ $m = (3+3)/2 = 3$

$\Rightarrow V > T[3] \Rightarrow (3 > 0) \Rightarrow bi=3+1=4$

$bi=4$ $bs=3$ stop : $bs < bi$, and val not found

	inf				sup			
T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8

m

	inf		sup					
T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8

m

			infsup					
T	-20	-4	0	5	8	73	142	201
	1	2	3	4	5	6	7	8

m

Binary search- algo

```
Algorithme recherche_dicho;
var
  v , m , inf , sup , pos : integer;
  T : array [100] d'integer;
begin
  // ici se trouve les instructions permettant de remplir le array T
  // ...
  inf ← 1; sup ← N, pos ← -1;
  tant inf < sup AND pos = -1 do
    m ← (inf + sup) div 2 ;
    si T[m] = v alors
      pos ← m;
    sinon
      si v < T[m] alors
        sup ← m - 1;
      sinon
        inf ← m + 1;
      fsi ;
    fsi ;
  fait;
  si pos = -1 alors
    write("La valeur ", v, " n'existe pas dans T");
  sinon
    write("La valeur ", v, " se trouve dans T à la position:", pos );
  fsi ;
```

Fin.

Binary search, C program

C Program version 1

```
#include <stdio.h>
int main(){
    int inf, sup, m, pos, v, N, i;
    // remplissage du array T
    printf("Entrez la taille du array : "); scanf("%d", &N);
    int T[N]; printf("Entrez les %d integers du array:\n", N);
    for (i = 0; i < N; i++) {
        printf("T[%d] = ", i); scanf("%d", &T[i]);
    }
    // lecture de V
    printf("donner une valeur à rechercher dans T: ");
    scanf("%d", &v); inf=0; sup=N-1; pos=-1;
    while (inf <= sup && pos==-1) {
        // calculer le milieu
        m = (sup + inf - 1) / 2;
        // vérifier si v est présent au milieu
        if (T[m] == v) pos=m;
        // si v est plus grand que v, alors ignorer la moitié gauche du array
        else if (T[m] < v) sup = m + 1;
        // si v est plus petit que v, alors ignorer la moitié droite du array
        else inf = m - 1;
    }
    // si pos est à -1 alors la valeur n'est pas trouvée
    if (pos == -1) printf("%d n'existe pas dans T", v);
    else printf("%d est présent à la position %d", v, pos);

    return 0;
}
```